SIMATS ENGINEERING
ASSESSMENT TOOL 2: Case Study

COURSE NAME:	Cloud Computing and Big Data Analytics	**COURSE CODE:**	CSA15

COURSE OUTCOME COVERED

CO3: *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications.* (BL3)

Assessment Tool Weightage: 20%

Dave's Taxonomy: Precision (Level 3) **Total Marks: 30**
Question Count: 3

Code of Conduct

I __M.Mary Sharlin (192511330)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the student: _M.Mary Sharlin

Assessment Tasks

Case Study

1. **Case Study 3: Online Food Ordering Platform**

A startup builds a cloud-based food ordering system accessible via web browsers and mobile clients. Frontend clients interact with backend services through RESTful Web APIs hosted on a cloud platform. Cloud storage is used to store menu images, invoices, and customer data securely. The infrastructure uses virtual machines and managed Kubernetes for scalability and reliability. Security is enforced through authentication tokens, HTTPS, and role-based access control. The system demonstrates integration of clients, APIs, storage, and platform services in real time.

ANSWER:

Case Study 3: Online Food Ordering Platform

1. Introduction

An **online food ordering platform** is a cloud-based application that allows customers to browse restaurants or menus, select food items, place orders, make payments, and track their orders using a **web browser or mobile application**.

The system uses **cloud computing** to provide scalability, storage, security, and reliable services. The frontend communicates with backend services through **RESTful Web APIs**. Cloud storage is used for storing menu images, invoices, and customer-related data.

2. System Architecture

The major components of the system are:

Customer → Web/Mobile Client → REST API → Backend Services → Database/Cloud Storage
Components

A. Frontend Clients

The frontend is the part of the application that customers directly interact with.

Examples:

- Web browser application
- Android application
- iOS application

Customers can:

- Create an account
- Login
- View food menus
- Search for food
- Add items to cart
- Place orders
- View invoices
- Track order status

The frontend sends requests to the backend using REST APIs.

B. RESTful Web APIs

REST (Representational State Transfer) APIs act as a communication layer between the frontend and backend.

For example:

- GET → Retrieve menu items
- POST → Place a new order
- PUT → Update order information
- DELETE → Remove an item or cancel an operation

Example flow:

Mobile App → REST API → Order Service → Database

When a customer places an order, the mobile application sends the order details to the REST API. The backend processes the request and stores the order information.

C. Backend Services

The backend contains the business logic of the application.

It handles:

- Customer registration and login
- Menu management
- Order processing
- Payment processing
- Invoice generation
- Order tracking
- Restaurant management
- Notifications

The backend can be divided into multiple services. For example:

User Service → Order Service → Payment Service → Notification Service

This makes the system easier to manage and scale.

3. Cloud Storage

Cloud storage is used to store large amounts of application data.

Examples include:

- Menu and restaurant images
- Customer information
- Digital invoices
- Order records
- Other application files

Instead of storing everything on a local computer, the system stores data in cloud infrastructure.

Advantages

- Large storage capacity

- Easy access from different locations
- Backup and recovery
- High availability
- Scalability
- Reduced need for local storage infrastructure

For example, when a restaurant uploads a food image, the image can be stored in cloud storage and its reference can be provided through the application.

4. Virtual Machines

Virtual Machines (VMs) are software-based computers that run on physical cloud servers.

The food ordering platform can use VMs to run:

- Backend applications
- REST API services
- Databases
- Supporting applications

Benefits of VMs

- Efficient use of physical hardware
- Isolation between applications
- Flexible resource allocation
- Easy deployment
- Ability to create multiple instances
- Improved reliability

For example, if one VM becomes overloaded, another VM can handle additional requests.

5. Kubernetes

Kubernetes is a container orchestration platform used to manage containerized applications.

In a food ordering system, different services can run in separate containers.

For example:

Kubernetes Cluster

```
    |
    ├── User Service
    ├── Menu Service
    ├── Order Service
    ├── Payment Service
    └── Notification Service
```

Kubernetes provides:

Scaling

If thousands of customers place orders during lunch time, Kubernetes can automatically increase the number of service instances.

Load Distribution

Customer requests can be distributed among multiple service instances.

Fault Recovery

If one container fails, Kubernetes can restart it automatically.

High Availability

Multiple instances of important services can run simultaneously.

Therefore, Kubernetes improves **scalability, availability, and reliability**.

6. Security Mechanisms

Security is very important because the platform handles customer accounts, orders, invoices, and potentially payment-related information.

A. Authentication Tokens

Authentication tokens are used to verify that a user is logged in.

Example:

Customer Login

```
    ↓
```

Server verifies credentials

↓
Authentication Token
↓
Customer sends token with requests
↓
Server verifies token
This prevents unauthorized users from accessing protected services.

B. HTTPS

HTTPS (HyperText Transfer Protocol Secure) encrypts communication between the customer's device and the server.

For example:

Mobile App
⇅
Encrypted HTTPS
⇅
Cloud Server

This helps protect sensitive information from being intercepted while it is being transmitted.

C. Role-Based Access Control (RBAC)

RBAC provides different permissions based on the user's role.

Role	Example Access
Customer	Browse menu and place orders
Restaurant	Manage menu and orders
Delivery Partner	View assigned deliveries
Administrator	Manage the complete system

For example, a customer should not be allowed to modify restaurant menus. RBAC prevents unauthorized operations.

7. Complete Working Flow

The complete operation can be explained as follows:

Step 1: User Opens Application

The customer opens the food ordering website or mobile application.

Step 2: Authentication

The customer logs in using their credentials. The backend verifies the user and provides an authentication token.

Step 3: Browse Menu

The application sends a request through the REST API to retrieve available food items.

Step 4: Display Menu

The backend retrieves menu information and images from cloud storage/database and sends the information back to the frontend.

Step 5: Place Order

The customer selects food items and places an order.

Step 6: Backend Processing

The REST API sends the order to the backend **Order Service**.

Step 7: Store Order

The order information is stored in the appropriate cloud database/storage.

Step 8: Generate Invoice

After successful order processing, an invoice can be generated and stored in cloud storage.

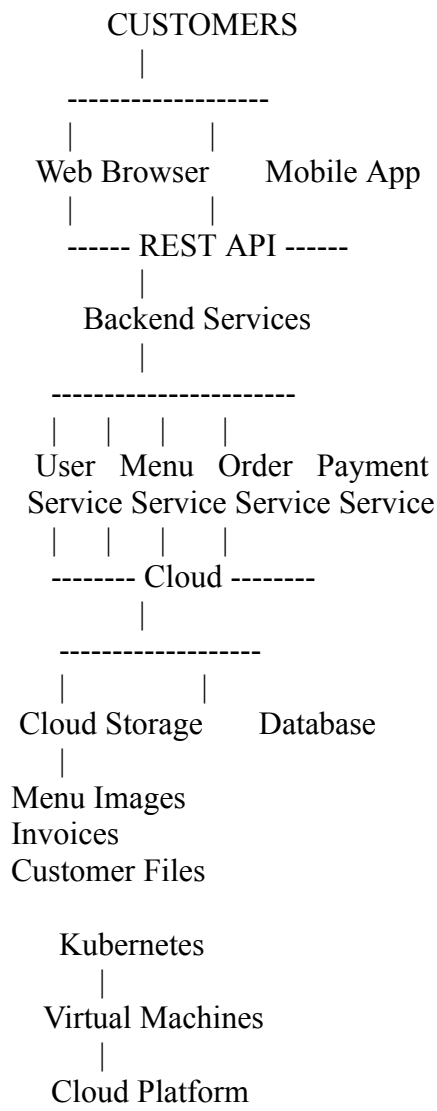
Step 9: Order Tracking

The customer can continuously request the order status through the REST API.

Step 10: Scaling

If the number of customers increases, Kubernetes can create additional service instances to handle the increased workload.

8. Overall Architecture



9. Advantages of the Cloud-Based System

1. **High Scalability** – The system can handle an increasing number of customers.
2. **High Availability** – Services can continue operating even if one instance fails.
3. **Cost Efficiency** – Cloud resources can be allocated according to demand.
4. **Secure Communication** – HTTPS protects data during transmission.
5. **Secure Access** – Authentication and RBAC prevent unauthorized access.
6. **Centralized Storage** – Images, invoices, and other files can be stored in cloud storage.
7. **Easy Maintenance** – Backend services can be updated independently.
8. **Real-Time Processing** – Orders and status updates can be processed quickly.
9. **Reliability** – Kubernetes can restart failed containers automatically.
10. **Accessibility** – Customers can access the platform through both web and mobile devices.

10. Example Scenario

Suppose **1,000 customers** order food during lunch time.
Initially, the system may have a few backend service instances running. As the number of requests increases:

Normal Traffic
↓
Few Service Instances
↓
Traffic Increases
↓
Kubernetes Detects Increased Load
↓
More Instances Created

↓
Requests Distributed
↓
System Continues Running Smoothly

After lunch, when the number of users decreases, unnecessary instances can be reduced. This is called **auto-scaling**.

11. Technologies and Their Roles

Technology	Role in Food Ordering System
Web Browser	Provides customer interface
Mobile App	Allows ordering through smartphones
REST API	Connects frontend and backend
Cloud Platform	Provides computing infrastructure
Virtual Machines	Run applications and services
Kubernetes	Manages and scales containers
Cloud Storage	Stores images and invoices
Database	Stores structured application data
Authentication Tokens	Verify users
HTTPS	Encrypts communication
RBAC	Controls access according to roles

12. Conclusion

The online food ordering platform is a good example of a **cloud-based distributed application**. Web and mobile clients communicate with backend services using **RESTful APIs**. Cloud storage provides scalable storage for images, invoices, and other files, while **virtual machines** provide computing resources. **Kubernetes** improves scalability and reliability by managing containers, distributing workloads, and restarting failed services. Security is maintained using **authentication tokens, HTTPS, and role-based access control**. Overall, the integration of **clients + REST APIs + backend services + cloud storage + VMs + Kubernetes + security mechanisms** creates a system that is **scalable, reliable, secure, and capable of handling real-time food ordering operations**.

Case Study Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Case	25%	Clearly identifies all technical and business requirements	Identifies most requirements	Basic understanding only	Poor understanding of case
Application of Concepts	25%	Applies suitable cloud concepts and tools effectively	Good application with minor issues	Partial application	Weak or incorrect application
Analysis	20%	Strong analysis with clear trade-off reasoning	Reasonable analysis	Limited analysis	Minimal analysis
Solution Design	20%	Solution is feasible, practical, and well justified	Reasonable solution with minor gaps	Basic solution	Unclear solution
Clarity	10%	Well-structured and easy to follow	Mostly clear	Somewhat unclear	Poorly organized